```
# Name : Prathamesh Lavange
# div : B
# Roll No :31
# PRN : 72332154L
# DMV PRATICAL NO 7
```

```
import kagglehub
import pandas as pd
import matplotlib.pyplot as plt
import os
import numpy as np
from statsmodels.tsa.arima.model import ARIMA

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
path = kagglehub.dataset_download(
    "jacksaleeby/s-and-p500-historical-data"
)

print("Dataset Path:", path)
print("Files:", os.listdir(path))
```

```
Downloading to /root/.cache/kagglehub/datasets/jacksaleeby/s-and-p500-historical-data/1.archive...
100%|██████████| 39.5M/39.5M [00:00<00:00, 174MB/s]Extracting files...

Dataset Path: /root/.cache/kagglehub/datasets/jacksaleeby/s-and-p500-historical-data/versions/1
Files: ['SP500_Historical_Data.csv']
```

```
csv_files = []

for root, dirs, files in os.walk(path):
    for file in files:
        if file.endswith(".csv"):
            csv_files.append(os.path.join(root, file))

print("CSV Files Found:")
print(csv_files[:10])

if not csv_files:
    raise FileNotFoundError("CSV file not found!")

file_path = csv_files[0]
df = pd.read_csv(file_path)

df.head()
```

```
CSV Files Found:
['/root/.cache/kagglehub/datasets/jacksaleeby/s-and-p500-historical-data/versions/1/SP500_Historical_Data.csv']
```

| | Ticker | Date | Open | High | Low | Close | Adj Close | Volume |
|---|---|---|---|---|---|---|---|---|
| **0** | A | 2000-01-03 | 47.07 | 47.18 | 40.27 | 43.04 | 43.04 | 4674353 |
| **1** | A | 2000-01-04 | 40.72 | 41.17 | 38.70 | 39.75 | 39.75 | 4765083 |
| **2** | A | 2000-01-05 | 39.60 | 39.75 | 36.05 | 37.28 | 37.28 | 5758642 |
| **3** | A | 2000-01-06 | 36.83 | 37.06 | 34.74 | 35.86 | 35.86 | 2534434 |
| **4** | A | 2000-01-07 | 35.30 | 39.41 | 35.27 | 38.85 | 38.85 | 2819626 |

```
print("Dataset Shape:", df.shape)
print("\nFirst 5 Rows:")
display(df.head())
```

```
Dataset Shape: (2703531, 8)

First 5 Rows:
```

| | Ticker | Date | Open | High | Low | Close | Adj Close | Volume |
|---|---|---|---|---|---|---|---|---|
| **0** | A | 2000-01-03 | 47.07 | 47.18 | 40.27 | 43.04 | 43.04 | 4674353 |
| **1** | A | 2000-01-04 | 40.72 | 41.17 | 38.70 | 39.75 | 39.75 | 4765083 |
| **2** | A | 2000-01-05 | 39.60 | 39.75 | 36.05 | 37.28 | 37.28 | 5758642 |
| **3** | A | 2000-01-06 | 36.83 | 37.06 | 34.74 | 35.86 | 35.86 | 2534434 |
| **4** | A | 2000-01-07 | 35.30 | 39.41 | 35.27 | 38.85 | 38.85 | 2819626 |

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2703531 entries, 0 to 2703530
Data columns (total 8 columns):
 #   Column     Dtype
---  ------     -----
 0   Ticker     object
 1   Date       object
 2   Open       float64
 3   High       float64
 4   Low        float64
 5   Close      float64
 6   Adj Close  float64
 7   Volume     int64
dtypes: float64(5), int64(1), object(2)
memory usage: 165.0+ MB
```

```
df.describe()
```

| | Open | High | Low | Close | Adj Close | Volume |
|---|---|---|---|---|---|---|
| **count** | 2.703531e+06 | 2.703531e+06 | 2.703531e+06 | 2.703531e+06 | 2.703531e+06 | 2.703531e+06 |
| **mean** | 7.276919e+01 | 7.362523e+01 | 7.189230e+01 | 7.277753e+01 | 7.277753e+01 | 8.716755e+06 |
| **std** | 1.478146e+02 | 1.494931e+02 | 1.461006e+02 | 1.478435e+02 | 1.478435e+02 | 4.919465e+07 |
| **min** | 3.000000e-02 | 3.000000e-02 | 3.000000e-02 | 3.000000e-02 | 3.000000e-02 | 0.000000e+00 |
| **25%** | 1.615000e+01 | 1.638000e+01 | 1.592000e+01 | 1.615000e+01 | 1.615000e+01 | 9.281085e+05 |
| **50%** | 3.399000e+01 | 3.440000e+01 | 3.357000e+01 | 3.399000e+01 | 3.399000e+01 | 2.211500e+06 |
| **75%** | 7.705000e+01 | 7.792000e+01 | 7.618000e+01 | 7.707000e+01 | 7.707000e+01 | 5.423200e+06 |
| **max** | 5.793920e+03 | 5.818240e+03 | 5.687240e+03 | 5.794840e+03 | 5.794840e+03 | 9.230856e+09 |

```
print("Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Ticker       0
Date         0
Open         0
High         0
Low          0
Close        0
Adj Close    0
Volume       0
dtype: int64
```

```
if "Ticker" in df.columns:
    stock = df[df["Ticker"] == "AAPL"].copy()
elif "ticker" in df.columns:
    stock = df[df["ticker"] == "AAPL"].copy()
else:
    raise ValueError("Ticker column not found!")

print("Selected Stock Data:")
display(stock.head())

print("Total AAPL Records:", len(stock))
```

```
Selected Stock Data:
```

| | Ticker | Date | Open | High | Low | Close | Adj Close | Volume |
|---|---|---|---|---|---|---|---|---|
| **11705** | AAPL | 2000-01-03 | 0.79 | 0.84 | 0.76 | 0.84 | 0.84 | 535796800 |
| **11706** | AAPL | 2000-01-04 | 0.81 | 0.83 | 0.76 | 0.77 | 0.77 | 512377600 |
| **11707** | AAPL | 2000-01-05 | 0.78 | 0.83 | 0.77 | 0.78 | 0.78 | 778321600 |
| **11708** | AAPL | 2000-01-06 | 0.79 | 0.80 | 0.71 | 0.71 | 0.71 | 767972800 |
| **11709** | AAPL | 2000-01-07 | 0.72 | 0.76 | 0.72 | 0.75 | 0.75 | 460734400 |

```
Total AAPL Records: 6573
```

```
if "Date" in stock.columns:
    date_column = "Date"
elif "date" in stock.columns:
    date_column = "date"
else:
    raise ValueError("Date column not found!")
```

```
stock[date_column] = pd.to_datetime(stock[date_column])

stock = stock.sort_values(date_column)
stock.set_index(date_column, inplace=True)

print("Date converted successfully!")
display(stock.head())
```

```
Date converted successfully!
```

| Date | Ticker | Open | High | Low | Close | Adj Close | Volume |
|---|---|---|---|---|---|---|---|
| **2000-01-03** | AAPL | 0.79 | 0.84 | 0.76 | 0.84 | 0.84 | 535796800 |
| **2000-01-04** | AAPL | 0.81 | 0.83 | 0.76 | 0.77 | 0.77 | 512377600 |
| **2000-01-05** | AAPL | 0.78 | 0.83 | 0.77 | 0.78 | 0.78 | 778321600 |
| **2000-01-06** | AAPL | 0.79 | 0.80 | 0.71 | 0.71 | 0.71 | 767972800 |
| **2000-01-07** | AAPL | 0.72 | 0.76 | 0.72 | 0.75 | 0.75 | 460734400 |

```
if "Close" in stock.columns:
    close_column = "Close"
elif "close" in stock.columns:
    close_column = "close"
else:
    raise ValueError("Closing price column not found!")

if "Volume" in stock.columns:
    volume_column = "Volume"
elif "volume" in stock.columns:
    volume_column = "volume"
else:
    volume_column = None

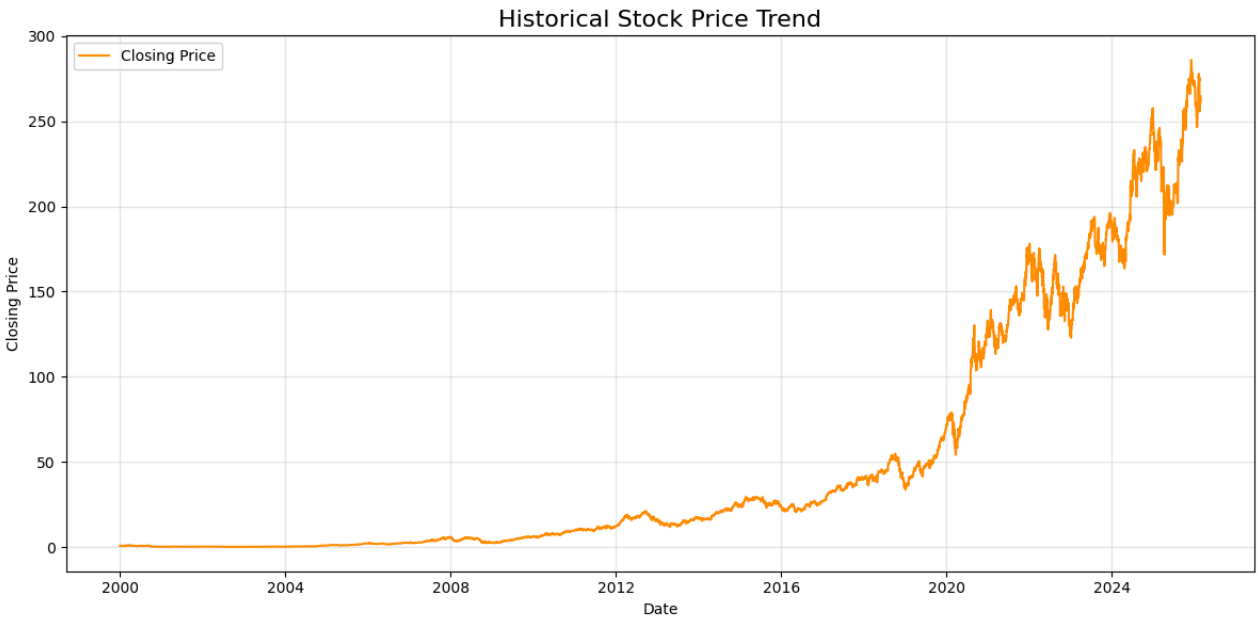
print("Closing Price Column:", close_column)
print("Volume Column:", volume_column)
```

```
Closing Price Column: Close
Volume Column: Volume
```

```
plt.figure(figsize=(12, 6))

plt.plot(
    stock.index,
    stock[close_column],
    color="darkorange",
    linewidth=1.5,
    label="Closing Price"
)

plt.title("Historical Stock Price Trend", fontsize=16)
plt.xlabel("Date")
plt.ylabel("Closing Price")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```




```
stock["MA_20"] = stock[close_column].rolling(window=20).mean()
stock["MA_50"] = stock[close_column].rolling(window=50).mean()

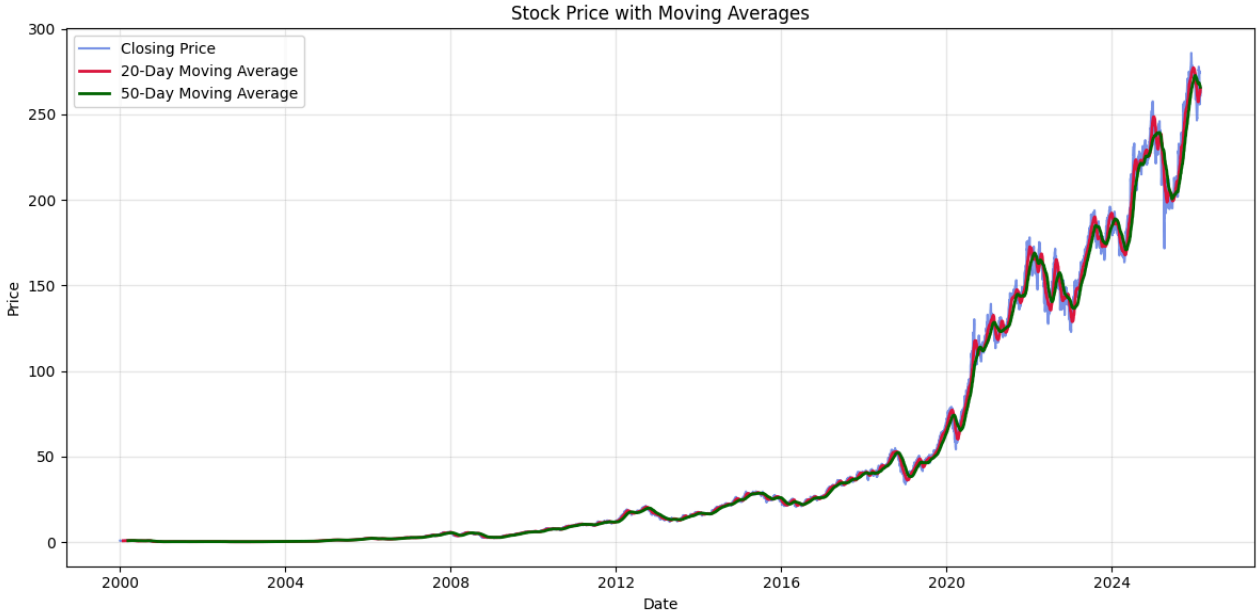
plt.figure(figsize=(12, 6))

plt.plot(
    stock.index, stock[close_column],
    color="royalblue", alpha=0.7,
    label="Closing Price"
)

plt.plot(
    stock.index, stock["MA_20"],
    color="crimson", linewidth=2,
    label="20-Day Moving Average"
)

plt.plot(
    stock.index, stock["MA_50"],
    color="darkgreen", linewidth=2,
    label="50-Day Moving Average"
)

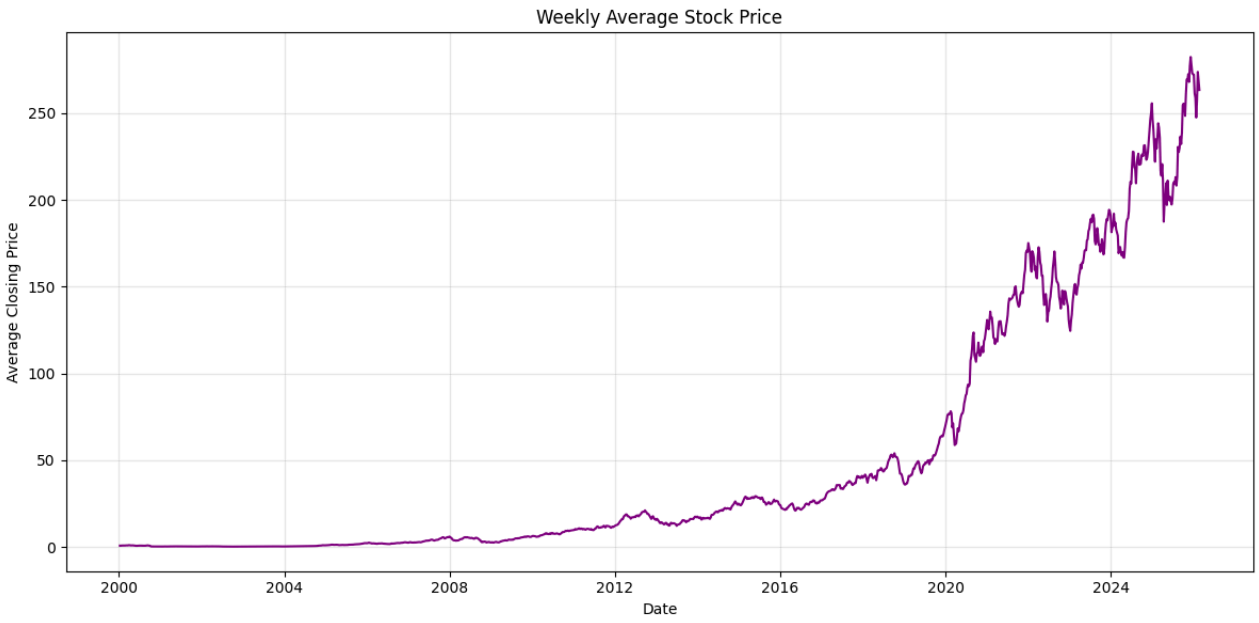
plt.title("Stock Price with Moving Averages")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```




```
weekly_price = stock[close_column].resample("W").mean()

plt.figure(figsize=(12, 6))
plt.plot(
    weekly_price.index,
    weekly_price.values,
    color="purple",
    linewidth=1.5
)

plt.title("Weekly Average Stock Price")
plt.xlabel("Date")
plt.ylabel("Average Closing Price")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

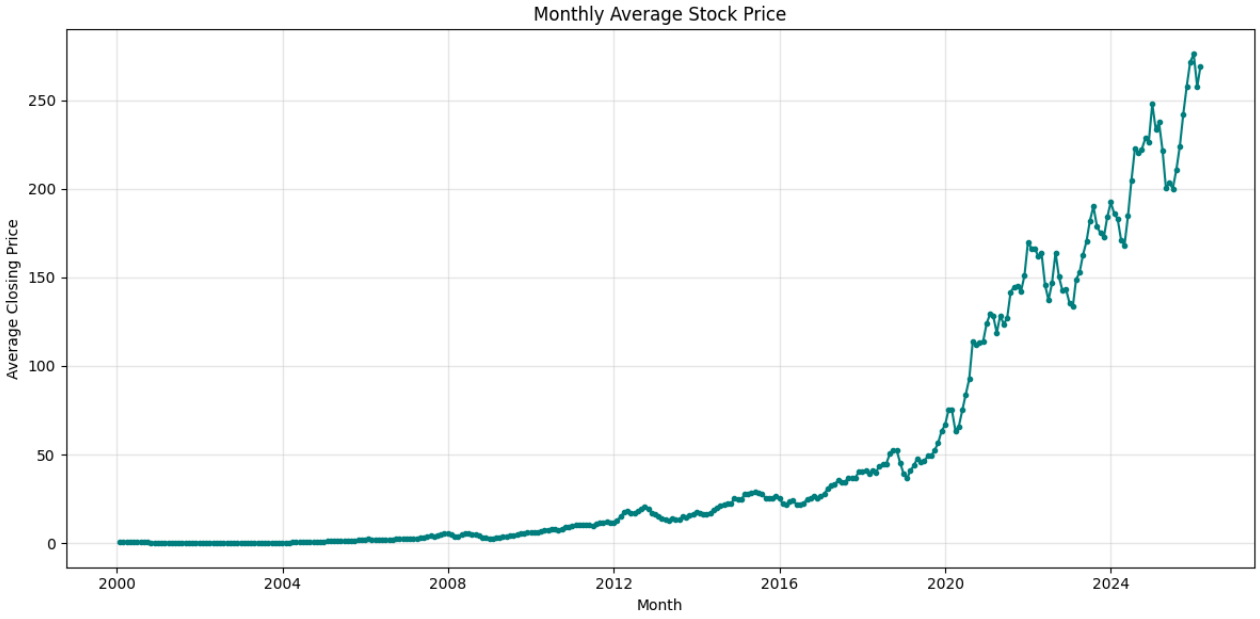


```
monthly_price = stock[close_column].resample("ME").mean()

plt.figure(figsize=(12, 6))

plt.plot(
    monthly_price.index,
    monthly_price.values,
    color="teal",
    marker="o",
    markersize=3
)

plt.title("Monthly Average Stock Price")
plt.xlabel("Month")
plt.ylabel("Average Closing Price")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

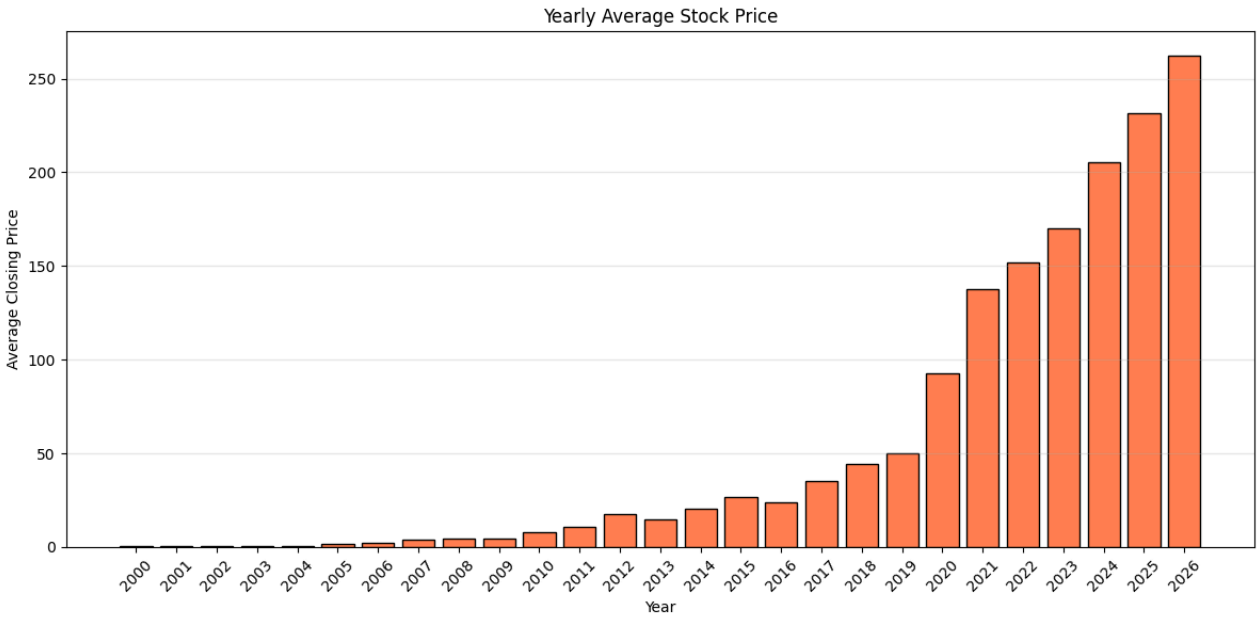



```
yearly_price = stock[close_column].resample("YE").mean()

plt.figure(figsize=(12, 6))

plt.bar(
    yearly_price.index.year.astype(str),
    yearly_price.values,
    color="coral",
    edgecolor="black"
)

plt.title("Yearly Average Stock Price")
plt.xlabel("Year")
plt.ylabel("Average Closing Price")
plt.xticks(rotation=45)
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```




```
series = pd.to_numeric(
    stock[close_column],
    errors="coerce"
).dropna()

series = series.sort_index()

# शेवटची 300 observations वापरा
series = series.tail(300)

values = series.to_numpy()

print("Total observations:", len(values))
display(series.head())
```

```
Total observations: 300
```

| Date | Close |
|---|---|
| **2024-12-09** | 245.41 |
| **2024-12-10** | 246.42 |
| **2024-12-11** | 245.15 |
| **2024-12-12** | 246.61 |
| **2024-12-13** | 246.78 |

**dtype:** float64

```
train_size = int(len(values) * 0.8)

train = values[:train_size]
test = values[train_size:]

print("Training data:", len(train))
print("Testing data:", len(test))
```

```
Training data: 240
Testing data: 60
```

```
model = ARIMA(train, order=(5, 1, 0))
model_fit = model.fit()

print("ARIMA model fitted successfully!")
```

```
ARIMA model fitted successfully!
```

```
forecast = model_fit.forecast(steps=len(test))

print("First 10 Forecasted Values:")
print(forecast[:10])
```

```
First 10 Forecasted Values:
[271.44077765 272.12109676 271.97561567 271.56703063 271.19826338
 271.05558405 271.01920421 271.08099192 271.15170829 271.19938521]
```

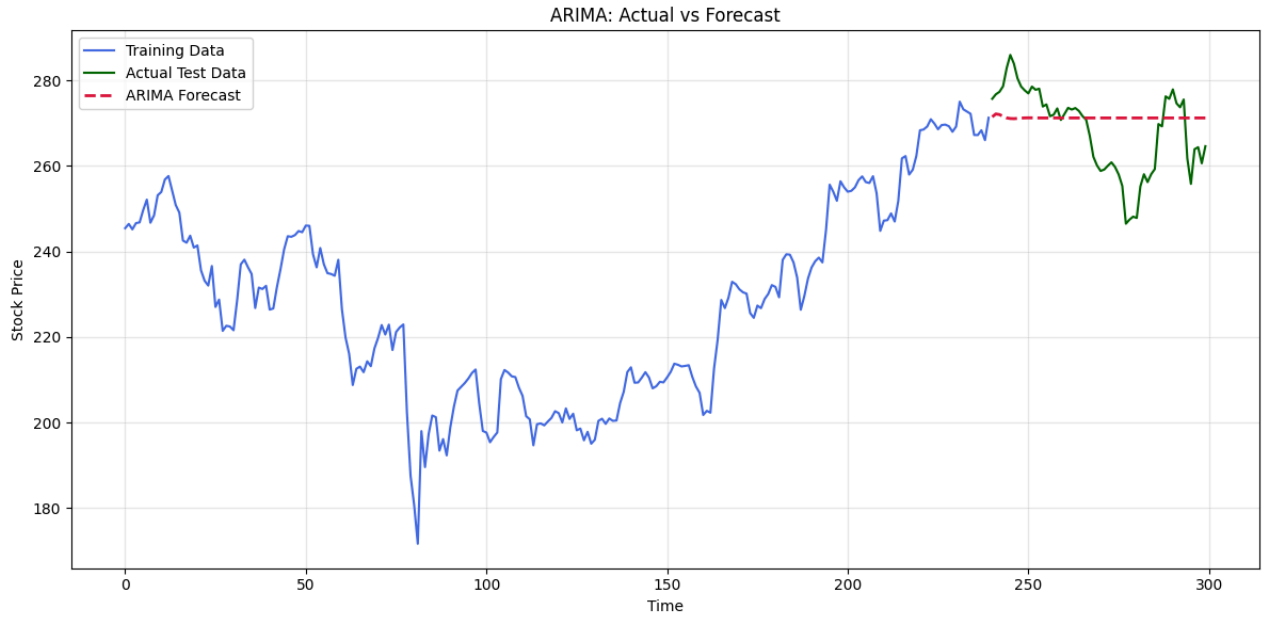
```
plt.figure(figsize=(12, 6))

plt.plot(
    range(len(train)),
    train,
    color="royalblue",
    label="Training Data"
)

plt.plot(
    range(len(train), len(values)),
    test,
    color="darkgreen",
    label="Actual Test Data"
)

plt.plot(
    range(len(train), len(values)),
    forecast,
    color="crimson",
    linestyle="--",
    linewidth=2,
    label="ARIMA Forecast"
)

plt.title("ARIMA: Actual vs Forecast")
plt.xlabel("Time")
plt.ylabel("Stock Price")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```




```
from statsmodels.tsa.arima.model import ARIMA

final_model = ARIMA(values, order=(5, 1, 0))
final_model_fit = final_model.fit()

future_forecast = final_model_fit.forecast(steps=30)
```

```
print("Next 30 Forecasted Stock Prices:")
```

```
Next 30 Forecasted Stock Prices:
[264.13187585 264.06909553 264.16184646 264.06025804 263.84883855
 263.85563277 263.83682102 263.84867005 263.87317922 263.88967919
 263.89379102 263.89463661 263.89136337 263.88796916 263.88597212
 263.88532929 263.88549461 263.88605243 263.88653239 263.88679505
 263.88685853 263.8868046  263.88671592 263.88664759 263.88661461
 263.88661083 263.88662206 263.88663581 263.88664531 263.88664917]
```

```
import pandas as pd
import matplotlib.pyplot as plt

last_date = series.index[-1]

future_dates = pd.bdate_range(
    start=last_date + pd.Timedelta(days=1),
    periods=30
)

plt.figure(figsize=(12, 6))

plt.plot(
    series.index,
    series.values,
    color="royalblue",
    label="Historical Stock Price"
)

plt.plot(
    future_dates,
    future_forecast,
    color="crimson",
    linestyle="--",
    marker="o",
    markersize=4,
    label="30-Observation ARIMA Forecast"
)

plt.title("Stock Price Forecast using ARIMA")
plt.xlabel("Date")
plt.ylabel("Stock Price")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

